

analytical_exercises/hmm_viterbi_training.py

```

1  #!/usr/bin/env python3
2  """
3  Minimal Viterbi-training script for Problem 2(c).
4
5  What it does:
6  1. Defines the HMM of Problem 2:
7     - States: 1, 2, 3
8     - Observations: H, T
9     - Given initial pi, A, B.
10  2. Uses the training sequence:
11     HHTTTHHTTTTHHHH
12  3. Runs several epochs of Viterbi-training:
13     (E-step: Viterbi, M-step: frequency re-estimation).
14  4. After EACH epoch, logs:
15     - best Viterbi path
16     - joint probability of path and observations
17     - updated pi, A, B
18  5. Writes everything into ONE file:
19     results/hmm_viterbi_training_log.txt
20  """
21
22  import os
23
24
25  def viterbi(pi, A, B, obs_seq, obs_map, state_labels):
26      """
27      Run the Viterbi algorithm for a single sequence.
28
29      Parameters
30      -----
31      pi : list[float]
32          Initial distribution, length N.
33      A : list[list[float]]
34          Transition matrix  $A[i][j] = P(q_t = j \mid q_{t-1} = i)$ .
35          Shape N x N.
36      B : list[list[float]]
37          Emission matrix  $B[j][k] = P(O_t = obs_k \mid q_t = j)$ .
38          Shape N x M, where M = number of observation symbols.
39      obs_seq : list[str]
40          Observation sequence (e.g. ['H','H','T',...]).
41      obs_map : dict[str, int]
42          Maps each observation symbol to a column index in B.
43      state_labels : list[int]
44          Human-readable labels for states (e.g. [1,2,3]).
45
46      Returns
47      -----
48      best_path_labels : list[int]
49          Most likely state sequence (MAP path) in terms of state labels.
50      best_prob : float
51          Joint probability of that path with the observations.

```

```

52     """
53     T = len(obs_seq)          # length of observation sequence
54     N = len(state_labels)     # number of states
55
56     # delta[t][j] = max probability of any path ending in state j at time t
57     # psi[t][j] = argmax previous state for that path (backpointer)
58     delta = [[0.0] * N for _ in range(T)]
59     psi = [[-1] * N for _ in range(T)]
60
61     # ----- Initialization (t = 0) -----
62     o0 = obs_map[obs_seq[0]]
63     for j in range(N):
64         delta[0][j] = pi[j] * B[j][o0]
65         psi[0][j] = -1 # no predecessor at t = 0
66
67     # ----- Recursion (t = 1, ..., T-1) -----
68     for t in range(1, T):
69         ot = obs_map[obs_seq[t]]
70         for j in range(N):
71             # For each destination state j, find best previous state i
72             best_val = -1.0
73             best_i = -1
74             for i in range(N):
75                 val = delta[t - 1][i] * A[i][j]
76                 if val > best_val:
77                     best_val = val
78                     best_i = i
79             # Multiply by emission probability for observation at time t
80             delta[t][j] = best_val * B[j][ot]
81             psi[t][j] = best_i
82
83     # ----- Termination -----
84     last_t = T - 1
85     best_last_state = max(range(N), key=lambda j: delta[last_t][j])
86     best_prob = delta[last_t][best_last_state]
87
88     # ----- Backtracking -----
89     best_path_indices = [best_last_state] * T
90     for t in range(last_t, 0, -1):
91         best_path_indices[t - 1] = psi[t][best_path_indices[t]]
92
93     # Convert internal indices (0..N-1) to external labels (1..N)
94     best_path_labels = [state_labels[idx] for idx in best_path_indices]
95     return best_path_labels, best_prob
96
97
98 def reestimate_parameters(path_labels, obs_seq, obs_map,
99                           num_states, num_obs,
100                           pi_old, A_old, B_old):
101     """
102     One M-step of Viterbi-training (hard EM).
103
104     Given:
105     - path_labels: Viterbi-decoded state sequence (labels like 1,2,3),

```

```

106     - obs_seq: observation sequence (e.g. ['H','T',...]),
107     treat them as fully observed and compute ML estimates:
108
109     pi_new[i] ~ frequency of starting in state i
110     A_new[i][j] ~ frequency of i->j transitions (normalized over j)
111     B_new[j][k] ~ frequency of symbol k in state j (normalized over k)
112
113     If a state never appears in the path (no counts),
114     we fall back to the OLD parameters for that state
115     to avoid degenerate rows.
116
117     Parameters
118     -----
119     path_labels : list[int]
120         State labels (1..num_states) for each time step.
121     obs_seq : list[str]
122         Observation sequence.
123     obs_map : dict[str, int]
124         Maps observation symbol -> index.
125     num_states : int
126         Number of states.
127     num_obs : int
128         Number of distinct observation symbols.
129     pi_old, A_old, B_old :
130         Old parameters (used as fallback if there are no counts).
131
132     Returns
133     -----
134     pi_new : list[float]
135     A_new : list[list[float]]
136     B_new : list[list[float]]
137     """
138     T = len(obs_seq)
139
140     # ----- Initial distribution pi -----
141     pi_new = [0.0] * num_states
142     start_state_idx = path_labels[0] - 1 # convert label (1..N) to index (0..N-1)
143     pi_new[start_state_idx] = 1.0
144
145     # ----- Transition counts -----
146     trans_counts = [[0] * num_states for _ in range(num_states)]
147     for t in range(1, T):
148         i = path_labels[t - 1] - 1 # from state index
149         j = path_labels[t] - 1 # to state index
150         trans_counts[i][j] += 1
151
152     # Normalize counts -> A_new.
153     # If a state i never appears (row_sum == 0), keep A_old[i].
154     A_new = [[0.0] * num_states for _ in range(num_states)]
155     for i in range(num_states):
156         row_sum = sum(trans_counts[i])
157         if row_sum > 0:
158             A_new[i] = [c / row_sum for c in trans_counts[i]]
159         else:

```

```

160         # No transitions from state i observed: keep old row.
161         A_new[i] = A_old[i][:]
162
163     # ----- Emission counts -----
164     emit_counts = [[0] * num_obs for _ in range(num_states)]
165     state_visit_counts = [0] * num_states
166     for t, label in enumerate(path_labels):
167         s_idx = label - 1          # state index
168         o_idx = obs_map[obs_seq[t]] # observation index
169         emit_counts[s_idx][o_idx] += 1
170         state_visit_counts[s_idx] += 1
171
172     # Normalize counts -> B_new.
173     # If a state j is never visited, keep B_old[j].
174     B_new = [[0.0] * num_obs for _ in range(num_states)]
175     for j in range(num_states):
176         total = state_visit_counts[j]
177         if total > 0:
178             B_new[j] = [c / total for c in emit_counts[j]]
179         else:
180             B_new[j] = B_old[j][:]
181
182     return pi_new, A_new, B_new
183
184
185 def log_params(f, header, pi, A, B):
186     """
187     Helper function to write parameters nicely into the log file.
188
189     Parameters
190     -----
191     f : file object
192         Open text file for writing.
193     header : str
194         A small header/title to print before the parameters.
195     pi : list[float]
196         Initial distribution.
197     A : list[list[float]]
198         Transition matrix.
199     B : list[list[float]]
200         Emission matrix (rows = states, cols = [H, T]).
201     """
202     f.write(header + "\n")
203     f.write("pi:\n")
204     f.write(" " + " ".join(f"{x:.6f}" for x in pi) + "\n\n")
205
206     f.write("A (rows = from state, cols = to state):\n")
207     for i, row in enumerate(A, start=1):
208         row_str = " ".join(f"{x:.6f}" for x in row)
209         f.write(f" from state {i}: {row_str}\n")
210     f.write("\n")
211
212     f.write("B (rows = states 1..3, cols = [H, T]):\n")
213     for j, row in enumerate(B, start=1):

```

```

214         row_str = " ".join(f"{x:.6f}" for x in row)
215         f.write(f" state {j}: {row_str}\n")
216     f.write("\n")
217
218
219 def main():
220     # ----- Basic settings -----
221     NUM_EPOCHS = 5 # number of Viterbi-training epochs (change if you want)
222
223     # Ensure output directory exists
224     os.makedirs("results", exist_ok=True)
225     log_path = os.path.join("results", "hmm_viterbi_training_log.txt")
226
227     # ----- HMM definition (Problem 2) -----
228     # States are labeled as 1, 2, 3
229     state_labels = [1, 2, 3]
230
231     # Observations: 'H' and 'T'
232     obs_symbols = ['H', 'T']
233     obs_map = {'H': 0, 'T': 1}
234
235     num_states = len(state_labels)
236     num_obs = len(obs_symbols)
237
238     # Initial distribution pi
239     pi = [1.0 / 3.0, 1.0 / 3.0, 1.0 / 3.0]
240
241     # Transition matrix A (rows = from-state, cols = to-state)
242     A = [
243         [0.8, 0.1, 0.1],
244         [0.2, 0.7, 0.1],
245         [0.2, 0.1, 0.7],
246     ]
247
248     # Emission matrix B (rows = states 1..3, cols = [H, T])
249     B = [
250         [0.5, 0.5], # state 1: P(H)=0.5, P(T)=0.5
251         [0.7, 0.3], # state 2: P(H)=0.7, P(T)=0.3
252         [0.25, 0.75], # state 3: P(H)=0.25, P(T)=0.75
253     ]
254
255     # Training observation sequence from Question (c)
256     obs_seq = list("HHTTTHHTTTTHHHH")
257
258     # ----- Open log file and write header -----
259     with open(log_path, "w", encoding="utf-8") as f:
260         f.write("Viterbi-training for HMM (Problem 2(c))\n")
261         f.write("=====\n\n")
262         f.write(f"Observation sequence: {''.join(obs_seq)}\n")
263         f.write(f"Number of epochs: {NUM_EPOCHS}\n\n")
264
265         # Log initial parameters (epoch 0)
266         log_params(f, header="Initial parameters (epoch 0):", pi=pi, A=A, B=B)
267

```

```

268 # ----- Viterbi-training epochs -----
269 for epoch in range(1, NUM_EPOCHS + 1):
270     # E-step: Viterbi decoding with current parameters
271     best_path, best_prob = viterbi(pi, A, B, obs_seq, obs_map, state_labels)
272
273     # M-step: update parameters using decoded path
274     pi_new, A_new, B_new = reestimate_parameters(
275         path_labels=best_path,
276         obs_seq=obs_seq,
277         obs_map=obs_map,
278         num_states=num_states,
279         num_obs=num_obs,
280         pi_old=pi,
281         A_old=A,
282         B_old=B,
283     )
284
285     # Overwrite parameters with updated ones
286     pi, A, B = pi_new, A_new, B_new
287
288     # Log results for this epoch
289     f.write(f"Epoch {epoch}\n")
290     f.write("-----\n")
291     f.write(f"Best Viterbi path (states): {best_path}\n")
292     f.write(f"Joint probability of best path: {best_prob:.10e}\n\n")
293
294     log_params(
295         f,
296         header=f"Parameters after epoch {epoch}: ",
297         pi=pi,
298         A=A,
299         B=B,
300     )
301
302     # ----- Final summary -----
303     f.write("Final parameters after all epochs:\n")
304     f.write("-----\n")
305     log_params(f, header="Final (same as last epoch): ", pi=pi, A=A, B=B)
306
307     # The log file now contains:
308     # - initial parameters
309     # - per-epoch path and probability
310     # - per-epoch parameters
311     # - final parameters
312
313
314 if __name__ == "__main__":
315     main()
316

```